



ABSTRACT

The idea for this project came from the *Tipsy cop and tipsy robber: Collisions of biased random walks on graphs* paper written by Pamela E. Harris, Erik Insko, and Florian Lehner in 2024 [1]. In their twist on the cops and robbers game, there is a cop and a robber which take turns moving around on a graph, each with a finite chance of moving randomly. We thought it would be interesting if the cop moved randomly every turn rather than just some. Because of this we created a game in which the cop, called a hunter in our game, always moves randomly as if they have no idea where the robber, in our game treasure, is located. A couple more aspects of the game were changed, since the hunter does not know where the treasure is there is no reason for it to move, so in our game it stays at it's starting vertex and only the hunter moves, thus the name buried treasure. We also changed the way the hunter moves so that there is always a finite expected capture time of the treasure. The hunter can move to any vertex that it has not already moved to that is a neighbor of a vertex it is on, or a neighbor of a vertex it has already been to. With this movement there will be a random walk of every connected graph in which all vertices will be traveled to exactly once. Since the hunter can only move randomly we focus on optimizing hunter and treasure starting positions. We expected a starting position on or near the center of the graph would minimize expected capture time. However our results showed that when the treasure was placed randomly, starting position for the hunter did not matter in regard to average expected capture time.

DEFINITIONS

- The **Buried Treasure Game**, played on a graph G , has a hunter who starts on $u \in V(G)$ and a buried treasure hidden on $v \in V(G)$. The hunter moves around the graph until it is on the vertex v , the starting position of the buried treasure. The hunter always moves randomly with equal probability to one of the neighbors of the vertex it is currently on, or one of the neighbors of a vertex it has previously traveled to. It can not travel to a vertex it has already traveled to.
- Suppose G is a graph with $u, v \in V(G)$. The **Expected Capture Time**, $EC(G, u, v)$, is the expected time for a hunter, starting on u , to find a treasure that is hidden on v when searching for buried treasure on G .
- The **Maximum Expected Capture Time** starting on u is defined as:

$$EC_{\max}(G, u) = \max_{v \in V(G)} \{EC(G, u, v)\}$$

- The **Average Expected Capture Time** starting on u is defined as:

$$EC_{\text{AVG}}(G, u) = \frac{\sum_{v \in V(G)} \{EC(G, u, v)\}}{|G|}$$

- The **Maximum Average Expected Capture Time** over all possible starting hunter positions is defined as

$$\bar{S}_{\max}(G) = \{u \in V(G) \mid EC_{\text{AVG}}(G, u) = \max_{v \in V(G)} EC_{\text{AVG}}(G, u)\}$$

- The **Minimum Average Expected Capture Time** over all possible starting hunter positions is defined as

$$\bar{S}_{\min}(G) = \{u \in V(G) \mid EC_{\text{AVG}}(G, u) = \min_{v \in V(G)} EC_{\text{AVG}}(G, u)\}$$

- The **Maximum Expected Capture Time** over all possible starting hunter positions is defined as

$$S_{\max}(G) = \{u \in V(G) \mid EC_{\max}(G, u) = \max_{v \in V(G)} EC_{\max}(G, u)\}$$

- The **Minimum Expected Capture Time** over all possible starting hunter positions is defined as

$$S_{\min}(G) = \{u \in V(G) \mid EC_{\max}(G, u) = \min_{v \in V(G)} EC_{\max}(G, u)\}$$

CODING A SIMULATION

- Random walks on graphs become very convoluted and nearly impossible to calculate by hand. Because of this we developed a code with that quickly calculates all of the parameters defined above given any set of edges and vertices that make up a connected graph.
- Below is an example output on a P_4 graph:
Smin (ties): Cop: 2 Robber: 4 Expected: 2.75 Cop: 3 Robber: 1 Expected: 2.75
Smax (ties): Cop: 1 Robber: 4 Expected: 3.0 Cop: 4 Robber: 1 Expected: 3.0
Savgmin (ties): Cop: 1 E: 1.5 Cop: 2 E: 1.5 Cop: 2 E: 1.5 Cop: 2 E: 1.5
Savgmax (ties): Cop: 1 E: 1.5 Cop: 2 E: 1.5 Cop: 2 E: 1.5 Cop: 2 E: 1.5
- After testing on many graphs we saw $\bar{S}_{\max}$ was always the same for every possible starting vertex. It's value only depended on n .

BURIED TREASURE GAME ON A P_4

After not seeing the $\bar{S}_{\max}$ values we expected in our simulation outputs we wanted to look closer at the math and calculate $\bar{S}_{\max}$ for some graphs by hand to verify. Below we look at a simple case of a P_4 graph.

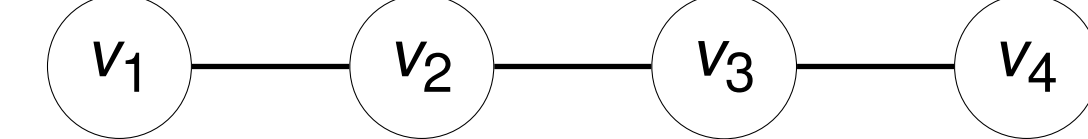
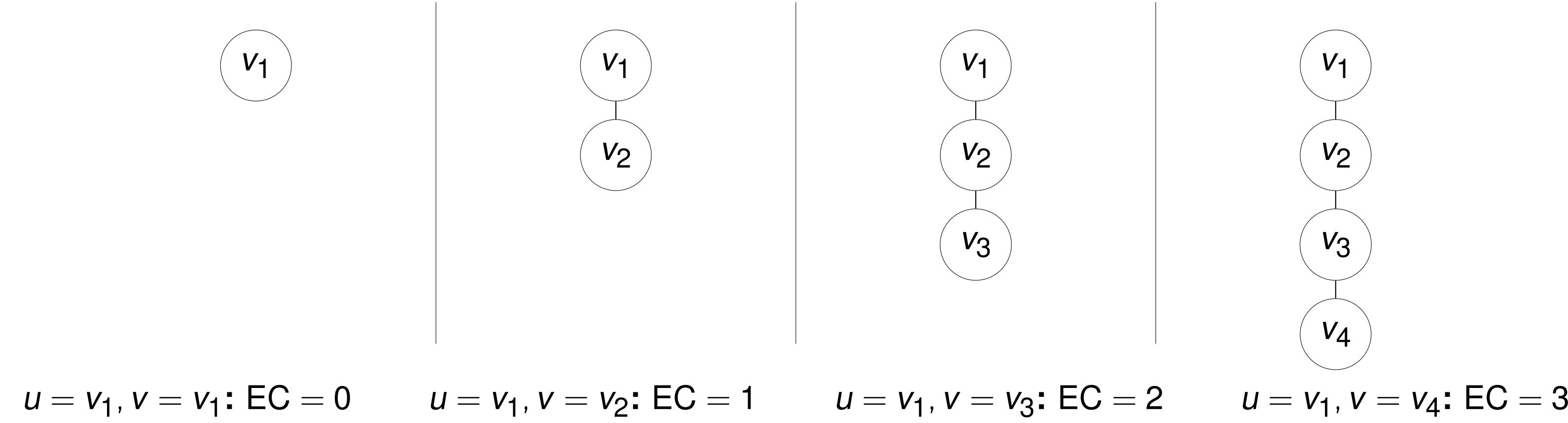


Figure: The path graph P_4 with labeled vertices.

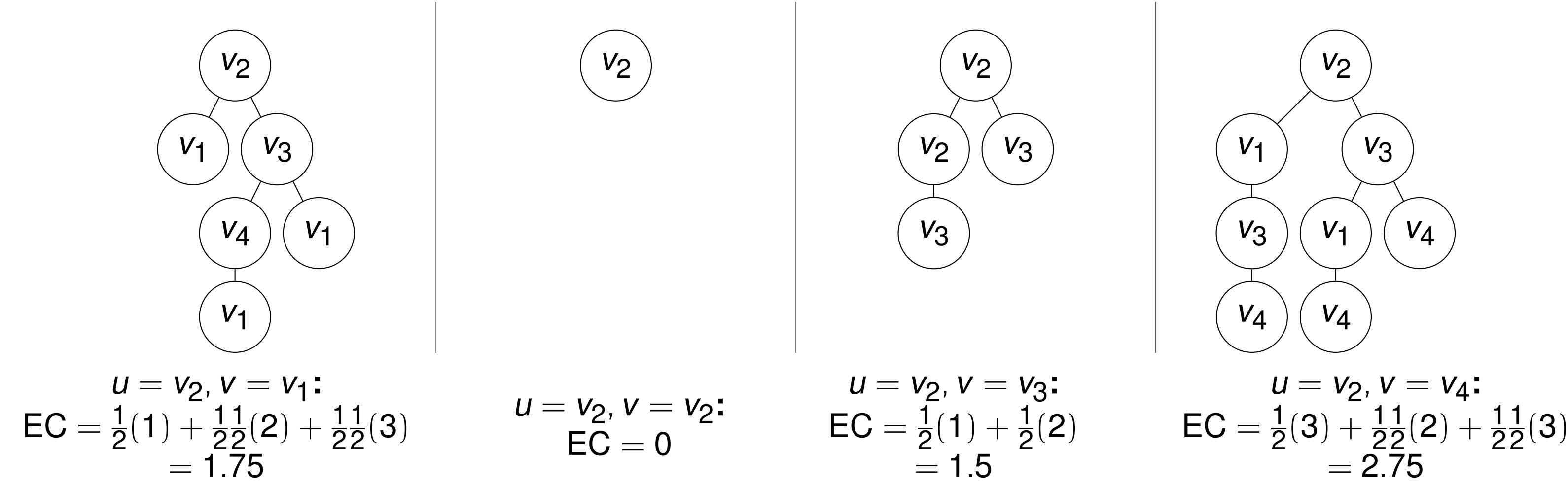
First we calculate average expected capture time for the case in which the hunter starts on v_1 . u is the starting hunter position, v is the starting treasure position.



Now that we have looked at all possible starting locations for v when $u = v_1$ we can calculate $EC_{\text{AVG}}(G, v_1)$:

$$\frac{0 + 1 + 2 + 3}{4} = 1.5$$

Next we calculate average expected capture time for the case in which the hunter starts on v_2 .



Now that we have looked at all possible starting locations for v when $u = v_2$ we can calculate $EC_{\text{AVG}}(G, v_1)$:

$$\frac{1.75 + 0 + 1.5 + 2.75}{4} = 1.5$$

Notice that if we rearrange our equation we get:

$$\frac{0 + (\frac{1}{2}1 + \frac{1}{2}1) + (\frac{1}{2}2 + \frac{1}{2}2 + \frac{1}{2}2) + (\frac{1}{2}3 + \frac{1}{2}3 + \frac{1}{2}3)}{4} = \frac{0 + 1 + 2 + 3}{4} = 1.5$$

As you can see $EC_{\text{AVG}}(G, v_1) = EC_{\text{AVG}}(G, v_2) = 1.5$. by symmetry, $EC_{\text{AVG}}(G, v_3) = EC_{\text{AVG}}(G, v_4) = 1.5$. This is equivalent to $\frac{(n-1)n}{2}$, since $n = 4$ in this example. This pattern will continue for any P_n when $u = v_1$ or $u = v_n$. As more vertexes are added to the path there will always be just one way for the hunter to get to them, consecutively adding numbers from 0 to $n - 1$.

GENERALIZING EC_{AVG} FOR ALL GRAPHS

At first we thought it was very strange that EC_{AVG} seemed to always be equal to $\frac{(n-1)n}{2}$, where $n = |V|$, regardless of graph orientation. After looking at the numbers from our P_4 demonstration however it made a bit more sense

- First notice that there will always be exactly one case, as long as $n > 0$, in which $EC = 0$, namely the case where $u = v$
- Also notice that there will always be at least one case in which the hunter finds the treasure in $0, 1, 2, \dots, n - 1$ turns for each hunter starting position. $n - 1$ will be the max number of turns to find the treasure because the hunter can not repeat vertices.
- Finally notice that in the P_4 example the probability being multiplied by each number of turns, when added up equals one. This is where we get the $\left(\frac{1}{2}\right)$ from, when adding up all the EC's.
- These observations lead us to the following proposition.

Proposition. $\Pr(\text{capt}(H) = k) = \frac{1}{n}$, Where $n = |V|$ and k is one of the possible capture times between 0 and $n - 1$ and H is a random hunt, or game, on the graph. $\text{capt}(H)$ is the number of turns that hunt took. Assuming this proposition is true we can prove that $EC_{\text{AVG}}(G, u)$, given G is connected and hunter starts on $u \in V(G)$, is $\left(\frac{n}{2}\right)\frac{1}{n}$:

$$\text{Proof. } EC_{\text{AVG}}(G, u) = \sum_{k=0}^{n-1} \Pr(\text{capt}(H) = k)k = \sum_{k=0}^{n-1} \frac{1}{n}k = \frac{1}{n} \sum_{k=0}^{n-1} k = \frac{(n-1)n}{2} \cdot \frac{1}{n} = \left(\frac{n}{2}\right)\frac{1}{n} \quad \square$$

PROOF OF PROPOSITION.

Each time the hunter moves it has a new set of vertices it can move to in the following turn. We will call the set of vertices it can travel to $O(n)$ where n is the number of turns the hunter has completed. When $n = 0$ the hunter has $\deg(u_{i_0})$ options to travel to next, all the neighbors of u_{i_0} , where u_{i_n} is vertex the hunter is on after n many turns. When the hunter has moved once it can still move to any of the neighbors of u_{i_0} , but also the neighbors of u_{i_1} . We get the equation $O(1) = \deg(u_{i_0}) + \deg(u_{i_1}) - 2$. The minus 2 is there because $\deg(u_{i_0})$ counts u_{i_1} which is not an option and $\deg(u_{i_1})$ counts u_{i_0} which is also not an option. Following the patten we get:

$$\begin{aligned} O(0) &= \deg(u_{i_0}) \\ O(1) &= \deg(u_{i_0}) + \deg(u_{i_1}) - 2 \\ O(2) &= \deg(u_{i_0}) + \deg(u_{i_1}) + \deg(u_{i_2}) - 4 \\ &\vdots \\ O(n) &= \deg(u_{i_0}) + \deg(u_{i_1}) + \deg(u_{i_2}) + \dots + \deg(u_{i_n}) - 2(n) \end{aligned}$$

Using these options we can calculate the $\Pr(\text{capt}(H) = k_n)$ where n goes from 0 to $|V| - 1$. We look at all possible ways the hunter can find the treasure in k turns, iterating through all possible options for each move. We sum together the probability of each of these walks happening and then multiply by the chance the treasure is on one of these vertices. This gives us the following sequence:

$$\Pr(\text{capt}(H) = k_1) = \left(\frac{1}{O(0)}\right) O(0) \cdot \frac{1}{n}$$

$$\Pr(\text{capt}(H) = k_2) = \sum_{i_1=1}^{O(0)} \left(\frac{1}{O(0)} \cdot \frac{1}{O(1)}\right) O(1) \cdot \frac{1}{n}$$

$$\Pr(\text{capt}(H) = k_3) = \sum_{i_1=1}^{O(0)} \sum_{i_2=1}^{O(1)} \left(\frac{1}{O(0)} \cdot \frac{1}{O(1)} \cdot \frac{1}{O(2)}\right) O(2) \cdot \frac{1}{n}$$

$$\vdots$$

$$\Pr(\text{capt}(H) = k_n) = \sum_{i_1=1}^{O(0)} \sum_{i_2=1}^{O(1)} \dots \sum_{i_{n-1}=1}^{O(n-2)} \left(\frac{1}{O(0)} \cdot \frac{1}{O(1)} \dots \frac{1}{O(n-2)} \cdot \frac{1}{O(n-1)}\right) O(n-1) \cdot \frac{1}{n}$$

Many of the terms in our sequence can now cancel out.

$$\begin{aligned} \Pr(\text{capt}(H) = k_n) &= \frac{1}{n} \sum_{i_1=1}^{O(0)} \sum_{i_2=1}^{O(1)} \dots \sum_{i_{n-1}=1}^{O(n-2)} \left(\frac{1}{O(0)} \cdot \frac{1}{O(1)} \dots \frac{1}{O(n-2)}\right) \\ &= \frac{1}{n} \cdot O(0) \sum_{i_2=1}^{O(1)} \dots \sum_{i_{n-1}=1}^{O(n-2)} \left(\frac{1}{O(0)} \cdot \frac{1}{O(1)} \dots \frac{1}{O(n-2)}\right) \\ &= \frac{1}{n} \cdot O(1) \cdot O(2) \dots O(n-2) \left(\frac{1}{O(1)} \dots \frac{1}{O(n-2)}\right) \\ &= \frac{1}{n} \end{aligned}$$

□

FUTURE WORK

There is a lot of work that still has to be done on this project. This poster focuses primarily on just one proof for the buried treasure game however there are many more things to look at, here are a few:

- Make the proof of our proposition more concise. Right now the proof is a bit abstract and could definitely be improved
- Look more at $S_{\max}$ and $S_{\min}$. Specifically figure out a generalization for finding each of them and prove it.
- Look for applications of this process of searching for treasure on a graph.
- Explore what happens when we look at all our parameters from the buried treasures position, keeping the treasure in the same location and moving hunter starting position around.

REFERENCES

- [1] Harris, Pamela E., Erik Insko, and Florian Lehner. "Tipsy Cop and Tipsy Robber: Collisions of Biased Random Walks on Graphs." *Theoretical Computer Science*, vol. 997, 2024, p. 114512.